1 Introduction to C++

1.1 Basic concepts:

C++ is a general-purpose programming language that was developed as an enhancement of the C language to include an object-oriented paradigm. It is an imperative and a compiled language.

Features of CPP:

- Object-Oriented Programming.
- Machine Independent.
- Simple.
- High-Level Language.
- Popular.
- Case-sensitive.
- Compiler Based.
- Dynamic Memory Allocation.

Features of OOP's:

- **Classes :**Class is blueprint of object.The basic structure of a class includes the class keyword followed by the class name and a block containing its members.
  - class MyClass {

    // Data members (attributes)

    Member Functions

    }
- **Objects**: An object is a Instance of Class.Objects are created by specifying the class name followed by the object name and optional parameters for constructors.

  ClassName objectName; // Object creation without parameters

  ClassName objectName(parameters);
- **Abstraction**: Abstraction is the process of hiding irrelevant information from the user.
- **.Encapsulation:**Data Hiding, Using access specifiers to control the visibility of class members, allowing the internal representation of the object to be hidden from the outside world.
- **Inheritance:**These concept of OOP's that allows a new class to inherit properties and behavior (attributes and methods) from an existing class. This existing class is referred to as the **base class** or parent class, and the new class is the **derived class** or child class.It helps Code Reusability ,Hierarchy and Specialization
- **Polymorphism:**Allows objects of different classes to be treated as objects of a common superclass. It enables a single interface to represent different underlying forms (types or classes). There are two main types of polymorphism in C++:

a. Function Overloading:Having multiple functions with the same name but different parameter types or numbers.

b. Operator Overloading: Redefining operators to work with user-defined types.

- **Dynamic Binding:**Dynamic binding, also known as late binding or runtime binding, is a concept in object-oriented programming (OOP) languages like C++. It's associated with polymorphism and occurs when the method or function to be executed for a given call is resolved at runtime rather than at compile time.
- **Message Passing:**

Message passing refers to a form of communication between objects in object-oriented programming (OOP). In OOP, objects interact with each other by sending and receiving messages, typically in the form of method calls.

Advantages of OOP's

- OOP is faster and easier to execute.
- OOP provides a clear structure for the programs.
- OOP helps to keep the C++ code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug.
- OOP makes it possible to create full reusable applications with less code and shorter development time.

DisAdvantages of OOP's:

- Size: Object-Oriented Programs are much larger than other programs.
- Effort: Object-Oriented Programs require a lot of work to create.
- Speed: Object-Oriented Programs are slower than other programs, because of their size.

Applications of OOP's:

- Real Time Systems.
- Client Server System.
- Hypertext and Hypermedia.
- Object Oriented Database.
- Neural Networks and Parallel Programming.
- AI Expert Systems.
- Simulation and Modeling System.
- Office Automation Systems.

Applications and features of C++

- System Software: C++ is widely used for developing operating systems, device drivers, and system software due to its low-level manipulation capabilities.
- Game Development: Many popular games, both small and large-scale, are built using C++ due to its performance and ability to control hardware resources.
- Embedded Systems: Its efficiency and ability to directly interact with hardware make it suitable for embedded systems in electronics, automobiles, and more.
- Financial Software: C++ is favored for high-frequency trading systems and financial applications where speed is crucial.
- Graphics and Animation: Applications involving computer graphics, animations, and rendering engines often utilize C++ for its performance.

1.3 Input and Output operator in C++

In C++, the input and output operators (<< and >>) are used for streaming data.

1. The <<(Insertion) operator is primarily used for output. It streams data to an output stream, such as the console, a file, or any output device.

```
#include <iostream>
using namespace std;
int main() {
   int num = 10;
   cout << "The value of num is: " << num << endl;
   return 0;
}
```

1. The >> (Extraction)operator is primarily used for input. It streams data from an input stream, such as the keyboard, a file, or any input device. For example:

```
#include <iostream>
using namespace std;
int main() {
   int num;
   cout << "Enter a number: ";
   cin >> num;
   cout << "You entered: " << num << endl;
   return 0;
}
```

In these examples, cout is the standard output stream, cin is the standard input stream, and endl is used to insert a newline character in the output stream. These operators allow C++ to handle input and output in a flexible and intuitive manner.

1.4 Simple C++ program

```
#include<iostream.h>
#include<conio.h>
int main() {
   // Declare variables
   int num1, num2, sum;
   // Input
   cout << "Enter first number: ";
   cin >> num1;
   cout << "Enter second number: ";
   cin >> num2;
   sum = num1 + num2;        // Calculate the sum
   cout << "The sum is: " << sum << endl;      // Output the result

   return 0;
}
```

2 Beginning with C++

2.1 Data type

In C++, data types define the type and size of data that a variable can hold. Here are some of the fundamental data types in C++:

Primitive Data Types:

Integer Types: Used to store whole numbers.
int,short,long,long long
Floating-Point Types: Used to store decimal numbers.
float,double,long double,
Character Types: Used to store single characters.
char
Boolean Type: Represents boolean values.
Bool
Derived Data Types:
Arrays: Collection of elements of the same data type.
Pointers: Variables that store memory addresses.
References: Alias to another variable.
User-Defined Data Types:
Structures: Grouping of variables under one name.
Classes: Blueprints for creating objects with their methods and properties.
Modifiers:
signed and unsigned,const,
volatile: Indicates that the variable may be changed unexpectedly by hardware or other threads of execution.
Enumerated Types:
enum: Defines an enumeration, allowing a programmer to define a new data type with a set of named constants.
Keywords:
Keywords in C++ are reserved words that have special meanings and purposes in the language. They cannot be used as identifiers (names for variables, functions, classes, etc.) because they are predefined for specific functionalities or operations. Here are some of the keywords in C++:

Keywords for Control Flow:

if: Used for conditional branching.
else: Used with if **to execute code when the** if **condition is false.**
switch: Used for multi-way branching based on different cases.
case: Used within switch **to define different cases.**
default: Used within switch **as a default case.**
while: Used to create a loop that executes a block of code as long as a condition is true.
do: Used in conjunction with while **to create a loop that executes a block of code at least**

once, then repeatedly as long as a condition is true.
for: Used to create a loop with a specified initialization, condition, and increment/decrement.
break: Used to exit from a loop or switch **statement.**
continue: Used to skip the current iteration of a loop and continue to the next iteration.
Keywords for Functions and Variables:
void: Indicates that a function does not return a value or that a pointer does not point to any type.
return: Used to return a value from a function.
const: Specifies that a variable's value cannot be changed.
static: Specifies that a variable or function has static storage duration.
extern: Indicates that a variable or function is defined in another file.
volatile: Indicates that a variable's value can be changed unexpectedly.
mutable: Allows modification of a const **class member in a** const **member function.**

Keywords for Data Types and Classes:
int, char, double, float, long, short: Data type keywords.
class: Used to define a class in C++.
struct: Defines a structure.
enum: Declares an enumeration.
typedef: Creates an alias for a data type.
typename: Often used in template programming.

Keywords for Memory Management and Pointers:
new: Allocates memory for objects or arrays.
delete: Deallocates memory allocated by new**.**
nullptr: Represents a null pointer.
this: Pointer to the current object within a member function.

2.2 Declaration of variables, dynamic initialization of variables, reference Variable
Declaration of Variables:
In C++, variable declaration involves specifying the data type and optionally initialising the variable.
int x; // Declaration of an integer variable 'x'
double pi = 3.14; // Declaration and initialization of a double variable 'pi'
char letter; // Declaration of a character variable 'letter'

Dynamic Initialization of Variables: Dynamic initialization involves assigning values to variables at runtime, often using input from the user or other dynamic sources.

```
int userInput;
cout << "Enter a number: ";
cin >> userInput; // Dynamic initialization based on user input

// Using the user-input value in calculations
int doubledValue = userInput * 2;
```

Reference Variables:

A reference variable in C++ is an alias or another name for an existing variable. Once a reference is established, any changes made to the reference variable affect the original variable and vice versa. Reference variables must be initialized when declared, and they cannot be null.

```
int original = 10;
int& refVar = original; // Declaration of a reference variable 'refVar' referring to 'original'

// Modifying the reference variable affects 'original'
refVar = 20;
cout << original; // Output will be 20
```

References are often used as function parameters to avoid unnecessary copying of data and to allow functions to modify the original values.

```
// Function that takes a reference as a parameter and modifies the original value
void increment(int& num) {
 num++;
}

int val = 5;
increment(val); // Passing 'val' by reference
cout << val; // Output will be 6
```

Using references can improve performance, readability, and enable functions to modify variables outside their scope.

Remember, reference variables must be initialized when declared, they cannot be reseated to refer to another variable after initialization, and they cannot be null.

2.3 Operators:

Arithmetic Operators:

+ (Addition),- (Subtraction),* (Multiplication),/ (Division),
% (Modulo): Returns the remainder after division.

Relational Operators:

== (Equal to): Checks if two operands are equal.
!= (Not equal to): Checks if two operands are not equal.
> (Greater than): Checks if the left operand is greater than the right operand.
< (Less than): Checks if the left operand is less than the right operand.
>= (Greater than or equal to): Checks if the left operand is greater than or equal to the right operand.
<= (Less than or equal to): Checks if the left operand is less than or equal to the right operand.

Logical Operators:

&& (Logical AND): Returns true if both operands are true.
|| (Logical OR): Returns true if at least one operand is true.

! (Logical NOT): Returns the opposite boolean value of the operand.
Bitwise Operators:
& (Bitwise AND): Performs bitwise AND on the operands.
| (Bitwise OR): Performs bitwise OR on the operands.
^ (Bitwise XOR): Performs bitwise XOR (exclusive OR) on the operands.
~ (Bitwise NOT): Flips the bits of the operand.
Assignment Operators:
= (Assignment): Assigns a value from the right operand to the left operand.
+=, -=, *=, /=, %=: Compound assignment operators (addition, subtraction, multiplication, division, modulo).
Increment/Decrement Operators:
++ (Increment): Increases the value of the operand by 1.
-- (Decrement): Decreases the value of the operand by 1.
Conditional Operator:
? : (Ternary): A conditional operator that evaluates an expression based on a condition.
Other Operators:
sizeof: Returns the size of a data type or variable in bytes.
, (Comma): Separates expressions, evaluates from left to right.

2.3.1 Scope resolution operator: The scope resolution operator :: is used to access members (variables or functions) within classes, namespaces, or global scope. It allows you to specify the context from which a particular member belongs.

2.3.2 Memory management operators

Memory management operators are used to dynamically allocate and deallocate memory during program execution. There are two main memory management operators: new and delete.

new Operator:new Operator: Allocates memory for a single variable or an array of variables on the heap. It returns the address of the allocated memory.

Syntax:
type* pointerName = new type; // Allocating memory for a single variable
type* arrayName = new type[size]; // Allocating memory for an array

delete Operator:

delete Operator: Deallocates memory that was previously allocated dynamically using new**. It releases the memory occupied by a single variable or an array.**

Syntax:
delete pointerName; // Deallocates memory for a single variable

delete[] arrayName; // Deallocates memory for an array

2.4 Manipulators:

In C++, manipulators are special functions or objects that modify the behavior of input/output streams. They're used with stream objects (like cin and cout) to format input and output in a specific

way. These manipulators are part of the <iomanip> header.Here are some commonly used manipulators:

1. Setw setw(int n): Sets the width of the next output field to n **spaces.**

```
#include <iostream>
#include <iomanip>
int main() {
 int num = 123;
 std::cout << std::setw(10) << num << std::endl; // Outputs: " 123"
 return 0;
}
```

2. setprecision

setprecision(int n): Sets the decimal precision for floating-point numbers to n **digits.**

```
#include <iostream>
#include <iomanip>

int main() {
 double pi = 3.1415926535;
 std::cout << std::setprecision(4) << pi << std::endl; // Outputs: "3.142"
 return 0;
}
```

3. fixed and scientific

fixed and scientific manipulators: Control the format of floating-point numbers.

```
#include <iostream>
#include <iomanip>
int main() {
 double num = 123456.789;
 std::cout << std::fixed << num << std::endl; // Outputs: "123456.789000"
 std::cout << std::scientific << num << std::endl; // Outputs: "1.234568e+05"
 return 0;
}
```

4. boolalpha

boolalpha: Outputs boolean values as "true" or "false".

```
#include <iostream>
#include <iomanip>
int main() {
 bool value = true;
 std::cout << std::boolalpha << value << std::endl; // Outputs: "true"
 return 0;
}
```

5. showpoint

showpoint: Always shows the decimal point and trailing zeros for floating-point numbers.

```
#include <iostream>
```

```
#include <iomanip>
int main() {
 double num = 123.4;
 std::cout << std::showpoint << num << std::endl; // Outputs: "123.400"
 return 0;
}
```

These manipulators offer flexibility in formatting output to suit specific requirements, making the presentation of data more readable and user-friendly. Manipulators can be combined and used together to achieve different formatting effects within C++ streams.

2.5 Functions:

In C++, functions are blocks of code that perform a specific task and can be called from other parts of the program. They're defined with a return type, a name, and can have parameters (inputs) and a body (the code to execute).

Here's a simple example of a function in C++:

```
#include <iostream>
// Function declaration or prototype
int add(int a, int b);
// Function definition
int add(int a, int b) {
   return a + b;
}
int main() {
   int x = 5, y = 3;
   int result = add(x, y); // Calling the add function
   std::cout << "The sum is: " << result << std::endl;
   return 0;
}
```

In this example:

add is the name of the function.

int before the function name indicates that the function returns an integer.

Inside the parentheses are the parameters (int a, int b) that the function takes. These act as placeholders for the values you'll pass when calling the function.

The function's body is enclosed in curly braces {} where the actual code to add two integers is written.

The return statement specifies the value that the function will give back when called.

2.5.1 Function prototyping, call by reference and return by reference

<u>Function Prototyping:</u>

Function prototypes provide a declaration of the function to the compiler before the function is defined or called. This informs the compiler about the function's name, return type, and parameters. It's particularly useful when functions are defined after the main() function or if they are used across multiple files.

Here's an example:

```
#include <iostream>
// Function prototype
```

```
int add(int a, int b);
int main() {
   int x = 5, y = 3;
   int result = add(x, y); // Function call
   std::cout << "The sum is: " << result << std::endl;
   return 0;
}
// Function definition
int add(int a, int b) {
   return a + b;
}
```

Call by Reference:

When you pass parameters by reference, you're passing the memory address of the variable rather than its value. This allows the function to directly modify the original variable.

Example:

```
#include <iostream>
void incrementByRef(int& num) {
   num++;
}
int main() {
   int x = 5;
   incrementByRef(x); // Passing x by reference
   std::cout << "Incremented value: " << x << std::endl; // Output will be 6
   return 0;
}
```

Return by Reference:

Returning by reference allows a function to return a reference to a variable. It can be used to avoid copying large objects and to manipulate the original value.

Example:

```
#include <iostream>
int& maxValue(int& a, int& b) {
   return (a > b) ? a : b;
}
int main() {
   int x = 5, y = 10;
   int& max = maxValue(x, y); // Returning reference
   max = 100; // Modifies the original value of x or y
   std::cout << "Updated value: " << x << ", " << y << std::endl; // Output will show the
updated value
   return 0;
}
```

These concepts are powerful tools in C++ that allow for more efficient memory usage and enable functions to interact with variables in different ways, whether it's through declaration, passing, or returning values.

2.5.2 Inline functions

Inline functions in C++ are a way to hint to the compiler that it should insert the entire body of the function wherever the function is called, rather than performing a regular function call. This is used to reduce the function call overhead and improve performance, especially for small functions.

To declare an inline function, you typically use the inline keyword before the function definition:

```
#include <iostream>
// Inline function declaration
inline int multiply(int a, int b) {
   return a * b;
}

int main() {
   int x = 5, y = 3;
   int result = multiply(x, y); // Inline function call
   std::cout << "The product is: " << result << std::endl;
   return 0;
}
```

When the compiler encounters an inline function call, it replaces the function call with the actual function body at compile time. However, the compiler might ignore the inline keyword suggestion if it deems it unnecessary, and ultimately, it's up to the compiler's discretion whether to inline a function or not.

Inline functions are handy for small functions, like accessors or mutators in classes, as they eliminate the overhead of a function call, but they might not be suitable for large or complex functions since inlining them could lead to larger executable size.

Remember, using inline is a suggestion to the compiler for optimization; the compiler might choose not to inline a function in certain circumstances, like recursive functions, functions with loops, or functions with too many conditional statements.

2.6 Default arguments

Default arguments in C++ allow you to provide default values for function parameters. When a function is called without providing values for these parameters, the default values are used. Here's an example:

```
#include <iostream>
// Function with default arguments
void greet(std::string name = "Guest", int age = 0) {
   std::cout << "Hello, " << name << "! Age: " << age << std::endl;
}
int main() {
   greet(); // Calling greet without arguments, default values used
   greet("Alice"); // Providing only the name, age defaults to 0
```

```
    greet("Bob", 30); // Providing both name and age
    return 0;
}
```

In this example:

The greet function has two parameters, name and age, both of which have default values ("Guest" and 0 respectively).
When greet() is called without any arguments, the default values are used for both name and age.
You can call greet with only one argument (e.g., greet("Alice")), and it will use the default value for the other argument.
Providing values for both parameters (greet("Bob", 30)) overrides the default values.
Default arguments are especially useful when you have functions that can be used in multiple contexts where certain parameters might often have the same value. They make functions more flexible and reduce the need for function overloading for different parameter combinations.

Assignment 1:

Two Marks Question

1. What is extraction and insertion operator?
2. Explain any two manipulators.
3. Define constructor.
4. What is inline function?
5. What is reference variable? What is its major use?
6. What is Abstraction and Encapsulation?
7. What is compile - Time polymorphism?
8. What is default argument?
9. What is the use of scope resolution operator?
10. What are the access specifies used in C++.
11. What is data abstraction?
12. What is return by reference?
13. Define :
14. Dynamic binding
15. Message Passing.
16. Explain structure of C++ program.
17. What is this pointer?
18. Explain const arguments.
19. Define pure virtual function.
20. Explain break, continue statements.
21. What is static data member?
22. What is setf() function ?
23. What is encapsulation?
24. What is void pointer?
25. Define the following terms :

Assignment 2:

26. Identifier
27. Constant.
28. What is function prototype?
29. Define destructor.
30. What is polymorphism?
31. What is virtual function?
32. Write uses of get( ) and put( ).
33. Define the following :
    1. tellg( )
    2. tellp( )
34. Define :
    1. class template
    2. Function template.
35. Define an object with an example.
36. Enlist user defined data types in C++
37. Differentiate between default and parameterized constructor.
38. Why do we make a function a pure virtual function?
39. List various classes available for file operations.
    Justify

40. “'Constructor can have different name as the class”.
41. What will be the output of the following
    cout<<setw(10)<<setfill(*)<<12345
42. a) Exception Handling
    b) Operator overloading
    c) Pointer to object with example.
43. Define abstract class
44. List the. operators that cannot be overloaded in C++.
45. What is inheritance? State its advantages.
46. What is Reference Variable ?
47. List any four features of OOPs.
48. What is an Abstract Class ?
49. What is Inline function ?
50. What is the difference between cin & getline( ) ?
51. What is generic pointer ?
52. What is stream concept in C++ ?
53. What is Dereferencing operator ?
54. Explain in which circumstances catch(....) statement would be used ?
55. Write seekg( ) function to :
    (i) move get pointer 10 bytes backward from end of file.
    (ii) move get pointer to start of file.